

NAME :- U.PRANEETH

ROLL NP:- 2520030188

SEC :- 03

-- Query 1: CREATE DATABASE

```
CREATE DATABASE IF NOT EXISTS bookflow7_db;  
USE bookflow7_db;
```

```
DROP TABLE IF EXISTS bank_transactions;
```

	#	Time	Action
	1	22:54:56	CREATE DATABASE IF NOT EXISTS bookflow7_db
	2	22:54:56	USE bookflow7_db
	3	22:54:56	DROP TABLE IF EXISTS bank_transactions

-- Query 2: CREATE TABLE

```
CREATE TABLE bank_transactions (  
    txn_id INT PRIMARY KEY,  
    customer_name VARCHAR(50),  
    branch_name VARCHAR(50),  
    transaction_type VARCHAR(20),  
    amount DECIMAL(10,2),  
    transaction_date DATE  
);
```

	4	22:54:56	CREATE TABLE bank_transactions (txn_id INT PRIMARY KEY, customer_name
---	---	----------	--

```
4 - commands* WEEK-3* AGGREGATE FUNCTION* x Joints*
Limit to 1000 rows
-- Query 3: Insert and Display
• INSERT INTO bank_transactions (txn_id, customer_name, branch_name, transaction_type, amount, transaction_date)
VALUES
    (101, 'Aswitha', 'Hyderabad', 'Deposit', 5000.00, '2024-01-05'),
    (102, 'Adhitha', 'Hyderabad', 'Withdrawal', 2000.00, '2024-01-06'),
    (103, 'Nihitha', 'Vijayawada', 'Deposit', 12000.00, '2024-01-08'),
    (104, 'Ananya', 'Vizag', 'Deposit', 8000.00, '2024-01-10'),
    (105, 'Rushika', 'Hyderabad', 'Withdrawal', 3500.00, '2024-01-11'),
    (106, 'Priyadarshini', 'Vizag', 'Deposit', 15000.00, '2024-01-12'),
    (107, 'Amulya', 'Vijayawada', 'Withdrawal', 1000.00, '2024-01-13'),
```

```
    (107, 'Amulya', 'Vijayawada', 'Withdrawal', 1000.00, '2024-01-13'),
    (108, 'Akshaya', 'Hyderabad', 'Deposit', 9000.00, '2024-01-14'),
    (109, 'Srujana', 'Vizag', 'Withdrawal', 4000.00, '2024-01-15'),
    (110, 'Varshini', 'Vijayawada', 'Deposit', 11000.00, '2024-01-16');
```

```
SELECT * FROM bank_transactions;
```

```
✓ 5 22:54:56 INSERT INTO bank_transactions (txn_id, customer_name, branch_name, transaction_type, amount, transaction_date)
```

```
-- Query 4: SUM()
```

```
SELECT SUM(amount) AS Total_Amount
FROM bank_transactions;
```

```
-- Query 5: AVG()
```

```
SELECT AVG(amount) AS Average_Transaction
FROM bank_transactions;
```

✓	5	22:54:56	SELECT * FROM bank_transactions (id, account_name, branch_name, transaction_type, amount) LIMIT 0, 1000
✓	6	22:54:56	SELECT * FROM bank_transactions LIMIT 0, 1000
✓	7	22:54:56	SELECT SUM(amount) AS Total_Amount FROM bank_transactions LIMIT 0, 1000
✓	8	22:54:56	SELECT AVG(amount) AS Average_Transaction FROM bank_transactions LIMIT 0, 1000

```
-- Query 6: MAX()
```

- SELECT MAX(amount) AS Highest_Transaction
FROM bank_transactions;

```
-- Query 7: MIN()
```

- SELECT MIN(amount) AS Lowest_Transaction
FROM bank_transactions;

✓	7	22:54:56	SELECT SUM(amount) AS Total_Amount FROM bank_transactions LIMIT 0, 1000
✓	8	22:54:56	SELECT AVG(amount) AS Average_Transaction FROM bank_transactions LIMIT 0, 1000
✓	9	22:54:56	SELECT MAX(amount) AS Highest_Transaction FROM bank_transactions LIMIT 0, 1000
✓	10	22:54:56	SELECT MIN(amount) AS Lowest_Transaction FROM bank_transactions LIMIT 0, 1000

```
-- Query 8: COUNT()
```

```
SELECT COUNT(*) AS Total_Transactions  
FROM bank_transactions;
```

```
-- Query 9: WHERE + SUM()
```

```
SELECT SUM(amount) AS Total_Deposit  
FROM bank_transactions  
WHERE transaction_type = 'Deposit';
```

✓	11	22:54:57	SELECT COUNT(*) AS Total_Transactions FROM bank_transactions LIMIT 0, 1000
✓	12	22:54:57	SELECT SUM(amount) AS Total_Deposit FROM bank_transactions WHERE transaction_type = 'Deposit' LIMIT 0, 1000

```
-- Query 10: GROUP BY
SELECT branch_name, SUM(amount) AS Total_Amount
FROM bank_transactions
GROUP BY branch_name;
```

```
-- Query 11: GROUP BY + HAVING
SELECT branch_name, SUM(amount) AS Total_Amount
FROM bank_transactions
GROUP BY branch_name
HAVING SUM(amount) > 20000;
```

✓	12	22:54:57	SELECT branch_name, SUM(amount) AS Total_Amount FROM bank_transactions GROUP BY branch_name;
✓	13	22:54:57	SELECT branch_name, SUM(amount) AS Total_Amount FROM bank_transactions GROUP BY branch_name;
✓	14	22:54:57	SELECT branch_name, SUM(amount) AS Total_Amount FROM bank_transactions GROUP BY branch_name;
✓	15	22:54:57	SELECT branch_name, SUM(amount) AS Total_Amount FROM bank_transactions GROUP BY branch_name;

```
-- Query 12: GROUP BY + ORDER BY
SELECT branch_name, SUM(amount) AS Total_Amount
FROM bank_transactions
GROUP BY branch_name
ORDER BY Total_Amount DESC;
```

```
-- Query 13: GROUP BY + HAVING + ORDER BY
SELECT branch_name, COUNT(*) AS Total_Transactions
FROM bank_transactions
```

```
FROM bank_transactions
GROUP BY branch_name
HAVING COUNT(*) >= 3
ORDER BY Total_Transactions DESC;
```


✓ 16 22:54:57 SELECT branch_name, COUNT(*) AS Total_Transactions FROM bank_transactions GRO

-- Query 14: WHERE + COUNT()

- SELECT COUNT(*) AS Withdrawals
FROM bank_transactions
WHERE transaction_type = 'Withdrawal';

-- Query 15: WHERE + AVG()

- SELECT AVG(amount) AS Avg_Deposit
FROM bank_transactions
WHERE transaction_type = 'Deposit';

✓ 17 22:54:57 SELECT COUNT(*) AS Withdrawals FROM bank_transactions WHERE transaction_type =

✓ 18 22:54:57 SELECT AVG(amount) AS Avg_Deposit FROM bank_transactions WHERE transaction_typ

-- Query 16: GROUP BY + MAX()

SELECT branch_name, MAX(amount) AS Highest_Amount
FROM bank_transactions
GROUP BY branch_name;

-- Query 17: GROUP BY + MIN()

SELECT branch_name, MIN(amount) AS Lowest_Amount
FROM bank_transactions
GROUP BY branch_name;

✓ 19 22:54:57 SELECT branch_name, MAX(amount) AS Highest_Amount FROM bank_transactions GRO

```
-- Query 18: WHERE + ORDER BY
```

```
SELECT *  
FROM bank_transactions  
WHERE amount > 8000  
ORDER BY amount DESC;
```

```
-- Query 19: GROUP BY + COUNT()
```

```
SELECT branch_name, COUNT(*) AS Customer_Count  
FROM bank_transactions
```

```
20 22:54:57 SELECT branch_name, MIN(amount) AS Lowest_Amount FROM bank_transactions
```

```
-- Query 20: HAVING + COUNT()
```

```
SELECT branch_name, COUNT(*) AS Total_Transactions  
FROM bank_transactions  
GROUP BY branch_name  
HAVING COUNT(*) > 3;
```

```
-- Query 21: GROUP BY + AVG()
```

```
SELECT branch_name, AVG(amount) AS Avg_Amount  
FROM bank_transactions  
GROUP BY branch_name;
```

#	Time	Action
21	22:54:57	SELECT * FROM bank_transactions WHERE amount > 8000 ORDER BY amount

```
-- Query 22: WHERE + GROUP BY + HAVING
SELECT branch_name, SUM(amount) AS Total_Deposit
FROM bank_transactions
WHERE transaction_type = 'Deposit'
GROUP BY branch_name
HAVING SUM(amount) > 15000;
```

```
-- Query 23: GROUP BY + HAVING + ORDER BY
SELECT branch_name, AVG(amount) AS Avg_Amount
FROM bank_transactions
```

```
126     FROM bank_transactions
127     GROUP BY branch_name
128     HAVING AVG(amount) > 7000
129     ORDER BY Avg_Amount DESC;
```

Setup

Result Grid			Filter Rows:	Export:	Wrap Cell Content:
	branch_name	Avg_Amount			
▶	Vizag	9000.000000			
	Vijayawada	8000.000000			

Result 27 ×

Output



Action Output

	#	Time	Action
✓	27	23:08:41	SELECT branch_name, AVG(amount) AS Avg_Amount FROM bank_transactions
✓	28	23:08:44	SELECT branch_name, AVG(amount) AS Avg_Amount FROM bank_transactions
✓	29	23:08:45	SELECT branch_name, AVG(amount) AS Avg_Amount FROM bank_transactions
✓	30	23:08:47	SELECT branch_name, AVG(amount) AS Avg_Amount FROM bank_transactions
✓	31	23:08:48	SELECT branch_name, AVG(amount) AS Avg_Amount FROM bank_transactions
✓	32	23:08:51	SELECT branch_name, AVG(amount) AS Avg_Amount FROM bank_transactions